

PyTorch 软件测试报告

1 测试概述

1.1 测试对象

本次测试对象为基于 PyTorch nn.Module 的 Hook 机制及其配套测试实现，重点验证模块级在前向传播、反向传播、异常处理、生命周期管理以及性能压力场景下的行为表现。本次测试不以某一具体业务模型的预测精度为评价目标，而是以 PyTorch 模块 Hook 机制的功能正确性、鲁棒性、兼容性和性能开销为主要评估对象。

测试内容围绕 forward pre-hook、forward hook、full backward hook 等典型 Hook 类型展开，主要关注以下方面：Hook 的注册、触发、执行顺序、移除是否符合预期；Hook 是否能够正确修改输入、输出或梯度；在异常输入、非法返回、递归调用、共享参数、设备迁移等边界场景下是否保持稳定；以及在不同 batch size、Hook 数量和训练/推理流程中，Hook 对运行时间和资源占用的影响。

1.2 测试人员

表 1 测试人员列表

姓名	学号	测试时间
刘彦	2352018	2026-05-20
毛经纶	2351867	2026-05-20
王晓涵	2351884	2026-05-20
李盛鹏	2351136	2026-05-20
吉稼熠	2352976	2026-05-20

2 测试方法

2.1 测试环境

表 2 测试环境列表

类别	配置/版本	说明
硬件	Windows 测试工作站；13th Gen Intel(R) Core(TM) i9-13900HX CPU；NVIDIA GeForce RTX 4060 Laptop GPU	用于功能测试、CPU benchmark、CUDA Hook 兼容性测试和 profiler 采样
操作系统	Windows 11 25H2	本地执行环境
Python/Conda	Python 3.10.18；Conda 25.5.1	用于运行测试与 benchmark
PyTorch/CUDA	PyTorch 2.5.1+cu121；CUDA 12.1	CUDA 用例在有 GPU 条件下执行
测试与绘图依赖	pytest 9.0.3；pluggy 1.6.0；numpy 1.26.4；matplotlib 3.10.6	pytest 用于功能断言，numpy/matplotlib 用于 benchmark 数据与图表

2.2 测试用例设计

本次测试用例依据当前已有测试内容进行设计，主要覆盖 `nn.Module` Hook 机制的基础功能、异常处理、边界场景、设备兼容性和性能观察。

等价类划分：按照 Hook 的作用阶段划分为 `forward pre-hook`、`forward hook`、`backward` 相关 Hook 等类别，分别验证其触发顺序、输入输出修改能力和梯度修改能力。当前基础功能测试已覆盖执行顺序、输入输出修改、异常传播、Hook 移除、`Backward Hook` 梯度修改和嵌套模块 Hook 顺序。

边界值分析：现有测试包含输入形状不匹配、不可调用对象注册、大量 Hook 注册与移除等边界情况；高级性能测试中包含 batch size 为 1、64、1024、4096、8192，以及 Hook 数量为 0、1、10、100、500、1000 的极端参数组合。

异常路径设计：现有测试覆盖 Hook 抛出异常、Hook 修改非法输入形状、注册不可调用对象等异常路径，用于验证异常能否被正确触发和传播。

场景法：现有测试包含嵌套模块、递归模块、共享参数模块、CUDA 设备运行、大量 Hook 移除、模型删除后的垃圾回收等场景，用于验证 Hook 在较复杂使用条件下的行为。

性能观察：现有高级 benchmark 覆盖极端 batch size 与 Hook 数量压力测试、`torch.compile` 兼容性对比、10 步模拟训练循环稳定性测试，以及 profiler 追踪文件生成。

表 3 测试用例列表

功能点/issue	测试用例 id	测试用例	优先级
Hook 执行顺序	TC-FWD-001	验证 <code>forward pre-hook</code> 、 <code>forward</code> 、 <code>forward hook</code> 的执行顺序。	最高
输入/输出修改	TC-FWD-002	验证 <code>pre-hook</code> 修改输入、 <code>post-hook</code> 修改输出后，结果与无 Hook 情况不同。	高
异常处理与非法注册	TC-ERR-001	验证 Hook 抛出异常、非法 shape、不可调用对象注册等异常场景是否正确报错。	高
Hook 生命周期管理	TC-LIFE-001	验证 Hook 移除、大量 Hook 清理以及模型删除后的垃圾回收情况。	高
Backward 梯度修改	TC-BWD-001	验证 <code>backward hook</code> 对 <code>grad_output</code> 和参数梯度的修改是否符合预期。	高
嵌套与递归调用	TC-STRUCT-001	验证父子嵌套模块、递归调用场景下 Hook 的触发顺序和触发次数。	高
共享参数场景	TC-BWD-002	验证共享 Linear 层被多次调用时， <code>backward hook</code> 对累计梯度的影响。	高
CUDA 兼容性	TC-CUDA-001	在 CUDA 可用时，验证模型与输入位于 GPU 后 Hook 仍能正常触发。	高
Forward/Backward 性能	TC-PERF-001	测量不同 batch size、Hook 数量下 <code>forward</code> 及 <code>forward+backward</code> 的耗时变化。	低
Pre/Post 与极限压力性能	TC-PERF-002	比较 <code>pre-hook</code> 、 <code>post-hook</code> 、 <code>both</code> 的开销，并覆盖 batch 1~8192、Hook 0~1000 的压力测试。	低
编译、训练循环与 Profiler	TC-PERF-003	对比 <code>torch.compile</code> ，测试 10 步训练循环稳定性，并生成 profiler 摘要与 trace 文件。	低

3 测试结果与分析

3.1 需求覆盖分析

本轮功能测试共收集 14 项 pytest 用例，执行结果为 14 passed，耗时 1.78s。性能 benchmark 与 profiler 测试均已完成，并生成对应图表、摘要文件和 trace 文件。

表 4 需求覆盖分析表

测试类别	对应用例	执行结果	备注
Forward hook 核心行为	TC-FWD-001、TC-FWD-002	通过	覆盖执行顺序、输入修改和输出修改
异常处理与非法输入	TC-ERR-001	通过	覆盖 Hook 抛异常、非法 shape、不可调用对象注册
Hook 生命周期管理	TC-LIFE-001	通过	覆盖 Hook 移除、大量 Hook 清理和垃圾回收
Backward hook 与梯度	TC-BWD-001、TC-BWD-002	通过	覆盖梯度缩放、共享参数和梯度数值验证
模块结构与控制流	TC-STRUCT-001	通过	覆盖嵌套模块顺序和递归调用次数
CUDA 设备兼容	TC-CUDA-001	通过	CUDA 可用时，GPU 场景下 Hook 正常触发
性能 benchmark 与 profiler	TC-PERF-001 至 TC-PERF-003	完成	生成 forward、backward、pre/post、extreme benchmark 图表和 profiler trace

3.2 测试覆盖分析

表 5 测试覆盖分析表

功能点/issue	测试用例数	执行数	通过/完成数
Forward hook 核心行为	2	2	2
异常处理与非法输入	3	3	3
Hook 生命周期与清理	3	3	3
Backward hook 与梯度	3	3	3
模块结构与控制流	2	2	2
CUDA 设备兼容	1	1	1
性能 benchmark 与 profiler	5	5	5
合计	19	19	19

现有测试覆盖了 nn.Module Hook 的基础行为、异常路径、Hook 生命周期、反向传播梯度修改、CUDA 设备兼容、递归调用、共享参数和垃圾回收等场景。性能测试覆盖了 forward、backward、pre/post Hook 开销、极限参数压力测试以及 profiler 追踪分析。尚未系统覆盖的内容包括 with_kwargs=True、prepend=True、多 Hook 注册顺序、多输出 backward hook 等场景。

3.3 缺陷统计

表 6 缺陷统计列表

Bug 描述	重要性等级	所属模块	是否修复	尚未修复	备注
本轮功能测试未发现被测代码缺陷	-	-	-	-	14 个用例全部通过

3.4 性能分析

基础 benchmark 结果显示，forward 与 backward 耗时会随 batch size 和 Hook 数量增加而上升。pre-hook、post-hook 与 both 三类组合均可正常运行，其中 both 场景同时注册 pre-hook 和 post-hook，整体开销高于单一 Hook 类型。基础 profiler 摘要显示，Self CPU time total 为 5.082ms；CPU total 主要集中在 aten::linear (4.545ms, 89.43%) 和 aten::addmm (4.455ms, 87.66%)，主要 CPU 内存分配来自线性层相关算子，aten::linear 和 aten::addmm 各约 800KB。

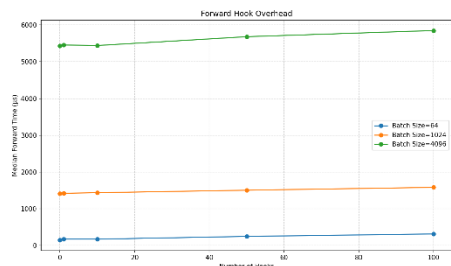


图 1 Forward Hook Overhead

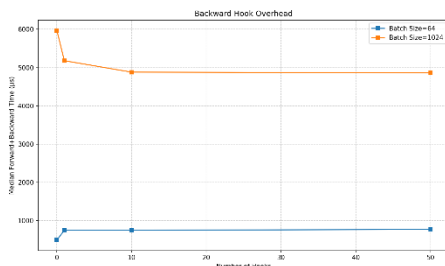


图 2 Backward Hook Overhead

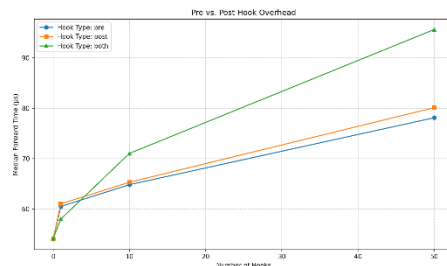


图 3 Pre vs Post Hook Overhead

高级 benchmark 覆盖了 batch size 为 1 至 8192、Hook 数量为 0 至 1000 的压力场景，并包含 torch.compile 兼容性对比、10 步训练循环稳定性测试和 profiler trace 导出。

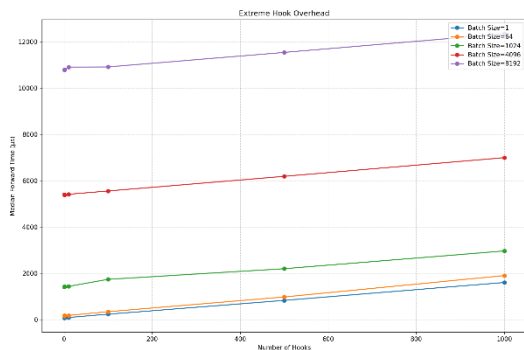


图 4 Extreme Hook Overhead

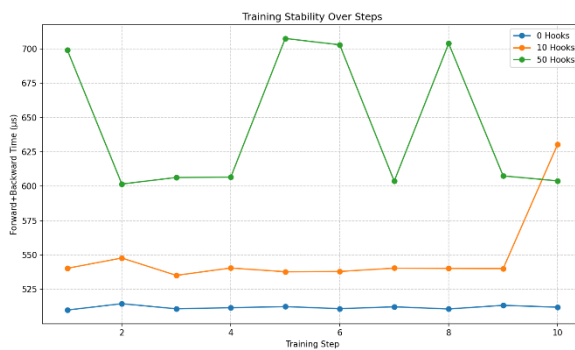


图 5 Training Stability Over Steps

高级 profiler 摘要显示，Self CPU time total 为 7.740ms；aten::linear CPU total 为 5.588ms，aten::addmm CPU total 为 5.471ms；反向传播阶段包含 AddmmBackward0、ReluBackward0、aten::mm、aten::threshold_backward 等算子。

4 测试结论

本轮测试在 PyTorch 2.5.1+cu121 环境下完成。功能测试共 14 项，全部通过；CUDA 条件测试实际执行通过。从测试结果看，当前测试套件能够验证 nn.Module Hook 的核心行为，包括执行顺序、输入输出修改、异常传播、Hook 移除、反向传播梯度修改、嵌套模块、递归调用、共享参数、CUDA 兼容和垃圾回收等内容。同时，性能 benchmark 能够观察 Hook 数量、batch size 和 Hook 类型组合对执行时间的影响。

后续可增加 with_kwargs、prepend、always_call、多 Hook 顺序、no_grad/inference_mode、train/eval 等语义覆盖；并为 benchmark 建立可比较的性能基线，便于后续版本回归测试。